Hackathon Problem Statement 2: AI-Powered Embedded Code Generator

Develop an AI agent or tool that transforms natural language prompts into production-ready embedded system code, including bare-metal firmware, peripheral drivers, or HAL implementations. Participants select the target architecture (ARM Cortex-M, AVR, RISC-V), microcontroller (e.g., STM32F4xx, ESP32, nRF52), or development board (e.g., Nucleo, Arduino Uno), then describe requirements like sensor interfaces (I2C MPU6050, SPI OLED, UART GPS) or functionality (e.g., "DMA-based ADC driver with interrupt handling for STM32F446RE and BMP280 pressure sensor").

Core Requirements

- **Prompt Parsing**: Extract MCU specs, peripherals, constraints (power, memory, RTOS), and output format from user input.

- **Code Generation**: Output compilable C/C++ code with register-level access, optimizations, error handling, and inline comments; support toolchains like STM32CubeIDE, PlatformIO, or Keil.

- **Documentation**: Auto-generate a step-by-step guide covering hardware setup, pin connections, compilation, flashing, and testing procedures.

- **Validation**: Include syntax checks, basic unit tests, or simulation hooks (e.g., for QEMU/Proteus).

Technical Guidelines

- Leverage LLMs (GPT-4o, Llama3, or local Ollama) with agentic frameworks like LangChain or AutoGen for multi-step reasoning (e.g., datasheet lookup → register config → code).

- Handle common peripherals: GPIO, UART/SPI/I2C, ADC/DAC, timers, DMA, interrupts.

- Support iteration: Users refine code via follow-up prompts (e.g., "Add low-power mode" or "Fix race condition").

Deliverables

1. Working prototype (CLI, web UI, or VS Code extension).

2. Demo with 3+ diverse prompts (e.g., sensor fusion, motor control, BLE stack).

3. Generated code + docs for a real-world use case.

4. GitHub repo with setup script.